

Exception Handling in C# with Examples

In this article, I am going to discuss **Exception Handling in C#** with Examples. This is one of the most important concepts in C#. As a developer, while developing an application, it is your key responsibility to handle the exception. Exception Handling is a procedure to handle the exception which occurred during the execution of a program. As part of this article, we are going to discuss the following pointers in detail.

1. **What are the Different Types of Errors?**
2. **What is an Exception in C#?**
3. **Who is responsible for abnormal termination of the program whenever runtime errors occur in the program?**
4. **What happens if an Exception is Raised in the program?**
5. **What CLR does when an Exception Occurred in the Program?**
6. **What is Exception Handling in C#?**
7. **Why do we need Exception Handling in C#?**
8. **What is the Procedure to Handle Exceptions in C#?**
9. **What is the Logical Implementation in C# to Handle Exception?**
10. **Exception handling in C# using Try Catch implementation**
11. **Properties of Exception Class in C#**

Types of Errors in C#

When we write and execute our code in the .NET framework then there is a possibility of two types of error occurrences. They are as follows:

1. **Compilation Errors**
2. **Runtime Errors**

Compilation Error in C#

The error that occurs in a program at the time of compilation is known as a compilation error (compile-time error). These errors occur due to syntactical mistakes in the program. That means these errors occur by typing the wrong syntax like missing double quotes in a string value and missing terminators in a statement, typing wrong spelling for keywords, assigning wrong data to a variable, trying to create an object for abstract class and interface, etc. So, whenever we compile the program, the compiler recognizes these errors and it will show us the list of errors.

So, in simple words, we can say that this type of error occurs due to a poor understanding of the programming language. These errors are identified by the compiler and can be rectified before the execution of the program only. So, these errors do not cause any harm to the program execution.

Runtime Error in C#

The errors which are occurred at the time of program execution are called runtime errors. These errors occurred at runtime due to various reasons such as when we are entering the wrong data into a variable, trying to open a file for which there is no permission, trying to connect to the database with the wrong user id and password, the wrong implementation of logic, and missing required resources, etc. So, in simple words, we can say that the errors which are come while running the program are called runtime errors.

Runtime errors are dangerous because whenever they occur in the program, the program terminates abnormally on the same line where the error gets occurred without executing the next line of code.

Note: The compiler will never check the logic, the compiler will only check the syntaxes. So, the compiler will identify the syntax error, but not the logical error.

Is Runtime Errors are Dangerous?

Yes, runtime errors are dangerous. See, if you are transferring the money, then there are two updated statements. One update statement will deduct the money from the source account and another update statement add the money to the destination account. Suppose, the first update statement was executed successfully and before executing the second update statement, some runtime error occurred because of some reason. That means the money is deducted from your account but not added to the destination account.

Then what you will do? You might be contacted with the bank, you might be going to the nearest bank and investigating what happens, Why the money is deducted from my account, and why it is not added to the destination account. This is the problem and this very dangerous and this is because we are not handling the runtime errors in our application.

What is an Exception in C#?

An Exception is a class in C# which is responsible for abnormal termination of the program when runtime errors occur while running the program. So, these errors (runtime) are very dangerous because whenever the runtime errors occur in the programs, the program gets terminated abnormally on the same line where the error gets occurred without executing the next line of code.

Note: Most people are saying Runtime Errors are Exceptions which is not true. Exceptions are classes that are responsible for abnormal termination of the program when runtime errors occur.

Who is Responsible for the Abnormal Termination of the Program whenever Runtime Errors occur?

Objects of Exception classes are responsible for abnormal termination of the program whenever runtime errors occur. These exception classes are predefined under BCL (Base Class Libraries) where a separate class is provided for each and every different type of exception like

1. **IndexOutOfRangeException**
2. **FormatException**
3. **NullReferenceException**
4. **DivideByZeroException**
5. **FileNotFoundException**
6. **SQLException,**
7. **OverflowException, etc.**

Each exception class provides a specific exception error message. All the above exception classes are responsible for abnormal termination of the program as well as they will be displaying an error message which specifies the reason for abnormal termination i.e. they provide an error message specific to that error.

So, whenever a runtime error occurs in a program, first the Exception Manager under the CLR (Common Language Runtime) identifies the type of error that occurs in the program, then creates an object of the Exception class related to that error and throws that object which will immediately terminate the program abnormally on the line where error got occur and display the error message related to that class.

Note: Exception class is the superclass of all Exception classes in C#.

What happens if an Exception is Raised in the Program in C#?

When an Exception is raised in C#, the program execution is terminated abnormally. That means the statements placed after the exception-causing statements are not executed but the statements placed before that exception-causing statement are executed by CLR.

What CLR does when an Exception Occurred in the program?

The CLR creates the exception class object that is associated with that logical mistake (exception) and terminates the program execution by throwing that exception object by using the throw keyword. So, we can say an exception is an event that occurs during the execution of a program that disrupts the normal flow of instruction execution. Let's understand this with an example.

Program Execution without Exception in C#

The following C# example shows program execution without exception. This is a very simple program, we are just dividing two numbers and printing the result on the console.

```
using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 20;
            int b = 10;
            int c;
            Console.WriteLine("A VALUE = " + a);
            Console.WriteLine("B VALUE = " + b);
            c = a / b;
            Console.WriteLine("C VALUE = " + c);
            Console.ReadKey();
        }
    }
}
```

Output:

```
A VALUE = 20
B VALUE = 10
C VALUE = 2
```

Program Execution with Exception in C#

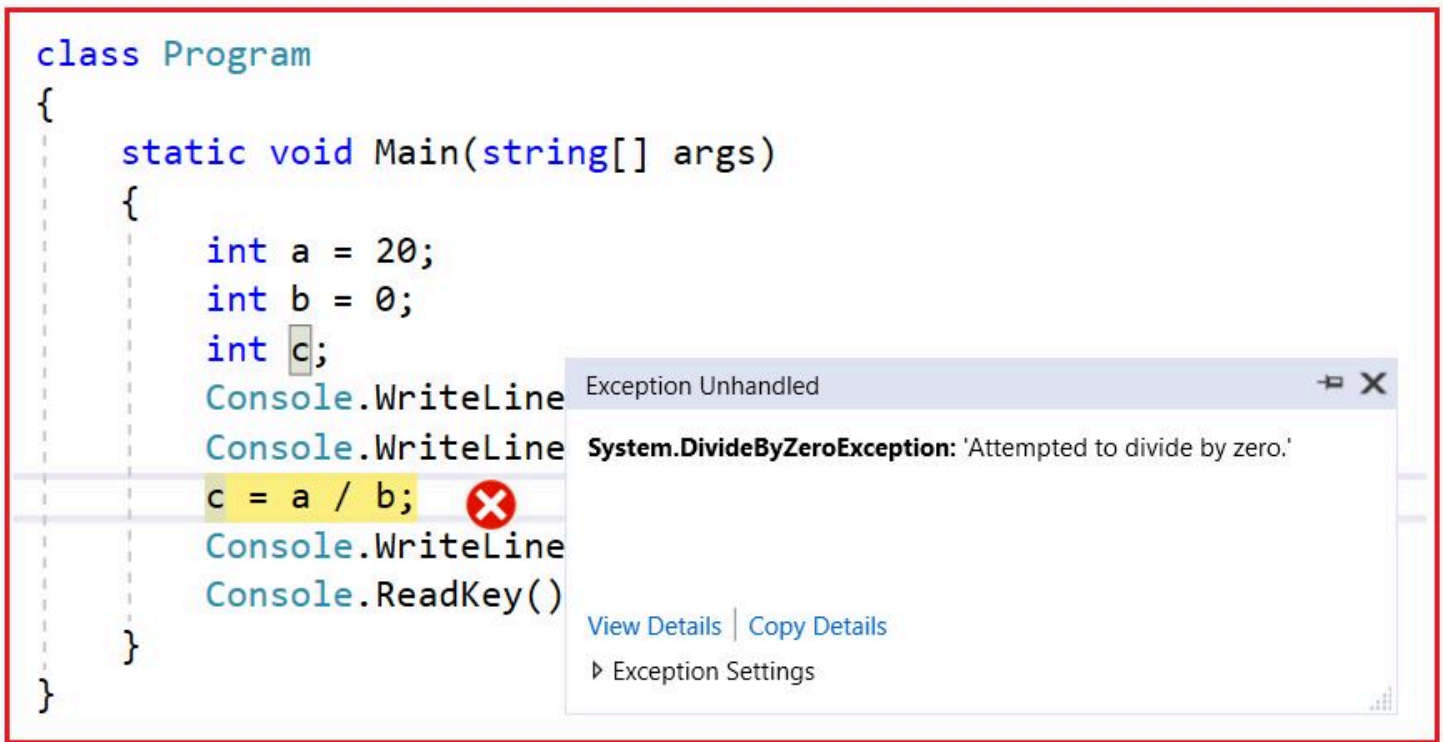
The following example shows program execution with an exception. As you can see, in the below code, we are dividing an integer number by 0 which is not possible in mathematics. So, it will throw the Divide By Zero Exception in this case. The statements which are present before the exception-causing statement i.e. before `c = a / b;` is executed and the statements which are present after the exception-causing statement will not be executed.

```
using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 20;
            int b = 0;
            int c;
            Console.WriteLine("A VALUE = " + a);
            Console.WriteLine("B VALUE = " + b);
            c = a / b;
            Console.WriteLine("C VALUE = " + c);
            Console.ReadKey();
        }
    }
}
```

Output:

```
A VALUE = 20
B VALUE = 0
```

After printing the above value it will give us the below error.



Explanation:

The CLR terminates the program execution by throwing **DivideByZeroException** because the logical mistake we committed here is dividing an integer number by integer zero. As we know it is not possible to divide an integer number by zero. So, what CLR will do in this case, first it will check what type of logical error is this. It will find that it will be a Divide By Zero Logical Error. So, then what CLR will do is, it will create an instance of **DivideByZeroException** class, and then it will throw that instance by using the throw statement like **throw new DivideByZeroException();** From the above program, we can define the exception technically as follows:

1. An exception is an event because when an exception is raised CLR internally executes some logic to prepare that exception-related messages.
2. The Exception is a signal because by looking into the exception message developer will take necessary actions against that exception.

Is the above Exception Message User Understandable?

Definitely, the answer is no. The user cannot understand the above exception message because they are .NET-Based exception messages. So the user cannot take any decision alone to resolve the above problem. A developer should guide to solve the above problem.

What is the Solution to the Above problem?

It is the developer's responsibility to convert .NET exception messages into user-understandable message formats. To solve this problem developer should handle the exception. Using the exception handling mechanism, the developer can catch the exception and can print and display user understandable messages.

What is Exception Handling in C#?

The process of catching the exception for converting the CLR given exception message to an end-user understandable message and for stopping the abnormal termination of the program whenever runtime errors are occurring is called Exception Handling in C#. Once we handle an exception under a program we will be getting the following advantages

1. We can stop the Abnormal Termination
2. We can perform any corrective action that may resolve the problem.
3. Displaying a user-friendly error message, so that the user can resolve the problem provided if it is under his control.

Why do we need Exception Handling in C#?

We need Exception Handling in C# because of the following two reasons.

1. To stop the Abnormal Termination of the program
2. To provide users with understandable messages when an exception is raised. So that users can make a decision without the developer's help.

Basically, by implementing Exception handling we are providing life to a program to talk to the user on behalf of a developer.

What is the Procedure to Handle Exceptions in C#?

The Exception Handling in C# is a 4 steps procedure

1. Preparing the exception object that is appropriate to the current logical mistake.
2. Throwing that exception to the appropriate exception handler.
3. Catching that exception
4. Taking necessary actions against that exception

How can we handle an Exception in .NET?

There are two methods to handle the exception in .NET

1. Logical Implementation
2. Try Catch Implementation

What is the Logical Implementation in C# to Handle Exception?

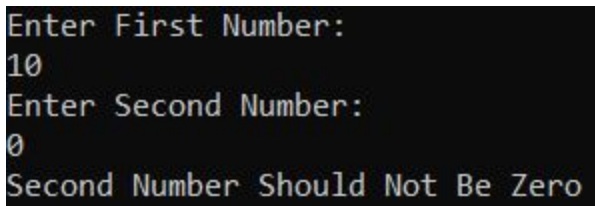
In logical Implementation, we need to handle the exception by using logical statements. In real-time programming, the first and foremost importance is always given to logical implementation only. If it is not possible to handle an exception using logical implementation then we need to go for try-catch implementation.

Handling Exceptions in C# using Logical Implementation

The following example shows how to handle exceptions in C# using the logical Implementation. Here, we are checking the second number i.e. variable Number2 value. If it equals 0, then we are printing one message saying the second number should not be zero else if the second number is not zero then we are performing our division operation and showing the results on the console. Here, we are using IF-ELSE logical statement to handle the exception.

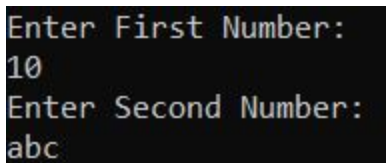
```
using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Number1, Number2, Result;
            Console.WriteLine("Enter First Number:");
            Number1 = int.Parse(Console.ReadLine());
            Console.WriteLine("Enter Second Number:");
            Number2 = int.Parse(Console.ReadLine());
            if (Number2 == 0)
            {
                Console.WriteLine("Second Number Should Not Be Zero");
            }
            else
            {
                Result = Number1 / Number2;
                Console.WriteLine($"Result = {Result}");
            }
            Console.ReadKey();
        }
    }
}
```

Output:



```
Enter First Number:
10
Enter Second Number:
0
Second Number Should Not Be Zero
```

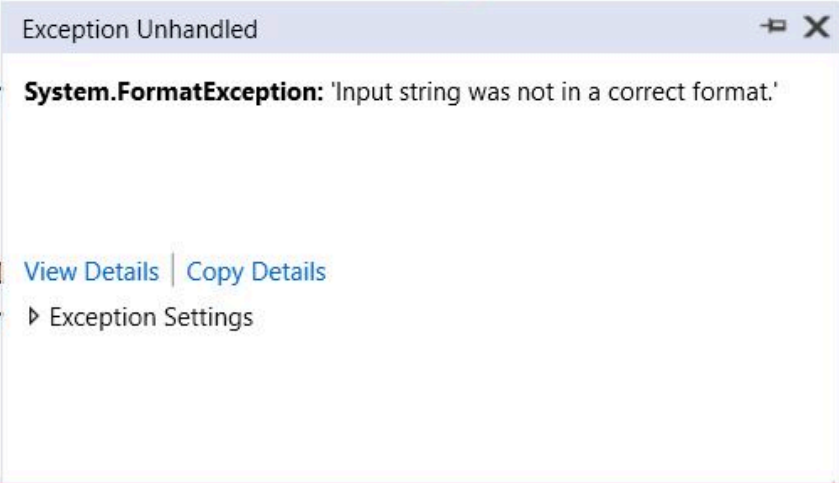
In the above case, when the user entered the second number as zero exception will be raised and that is handled using the logical implementation in C#. But while we are entering two numbers instead of the number if we entered any character then it will give you one exception which is **FormatException** which is not handled in this program as shown below.



```
Enter First Number:
10
Enter Second Number:
abc
```

Here we entered the second value as “abc”. So, it will give us the below exception.

```
static void Main(string[] args)
{
    int Number1, Number2, Result;
    Console.WriteLine("Enter First Number:");
    Number1 = int.Parse(Console.ReadLine());
    Console.WriteLine("Enter Second Number:");
    Number2 = int.Parse(Console.ReadLine());
    if (Number2 == 0)
    {
        Console.WriteLine("Second Number cannot be zero.");
    }
    else
    {
        Result = Number1 + Number2;
        Console.WriteLine("Sum of two numbers is: " + Result);
    }
    Console.ReadKey();
}
```



So to handle such types of exceptions in C# we need to go for Try catch implementation. So, the point that you need to remember is if we are unable to handle the exception using logical implementation, then only we need to go for try-catch implementation in C#.

Exception handling in C# using Try Catch implementation

To implement the try-catch implementation, the .NET framework provides three keywords. They are as follows:

1. **Try**
2. **Catch**
3. **finally**

Try Block:

The try keyword establishes a block in which we need to write the exception causing and its related statements. That means exception-causing statements and the related statements which we should not execute when an exception occurred must be placed in the try block. When the exception occurred, the CLR will create an instance of the Exception class based on the logical error and then throw that Exception object which is going to be handled by the corresponding catch block.

Catch Block:

The catch block is used to catch the exception that is thrown from its corresponding try block. It has the logic to take necessary actions on that caught exception. The Catch block syntax in C# looks like a constructor. It does not take accessibility modifiers, normal modifiers, or return types. It takes only a single parameter of type Exception or any child class of the parent Exception class. Inside the catch block, we can write any statement which is legal in .NET including raising an exception. In this case, we can stop the abnormal termination of the program and we can also give user understandable error message so that the user can take necessary action to resolve the error.

Finally Block:

The keyword finally establishes a block that definitely executes the statements placed in it irrespective of whether any exception has occurred or not. That means the statements that are placed in finally block are always going to be executed irrespective of whether any exception is thrown or not, irrespective of whether the thrown exception is handled by the catch block or not.

Syntax to use Exception Handling in C#:

The following image shows the syntax to use exception handling in C#. It starts with the try block, followed by the catch block, and writing the finally block is optional. You can write any number of catch blocks for a given try block in C#. This will handle different types of exceptions thrown by the try block.

```
void SomeMethod()
{
    //Normal Statements
    try
    {
        //Exception causing and its related statement which
        //we don't want to execute when exception occurred
    }
    catch (SomeException1 one)
    {
        //Statements which required to execute when SomeException1
        //Occurred in the program
    }
    catch (SomeException2 two)
    {
        //Statements which required to execute when SomeException2
        //Occurred in the program
    }
    catch (Exception generic)
    {
        //This is generic exception handling block. If non of the above
        //Exception catch block handles the exception, then this catch
        //block is going to be execute
    }
    //Normal Statements
}
```

Once we use the try and catch blocks in our code the execution takes place as follows:

1. If all the statements under the try block are executed successfully, from the last statement of the try block, the control directly jumps to the first statement that is present after the catch block (after all catch blocks) without executing the catch block (it means there is no runtime error in the code at all).

2. If any of the statements in the try block causes an error, from that statement without executing any other statements in the try block, the control directly jumps to the catch blocks which can handle that exception.
3. If a proper catch block is found that handles the exception thrown by the try block, then the abnormal termination stops there, executes the code under the catch block, and from there again it jumps to the first statement after all the catch blocks.
4. If a matching catch block is not found, then the generic catch block is going to execute to handle the abnormal termination.
5. If you don't have the generic catch block and if any of the catch blocks are unable to handle the exception, then again, the program execution terminates abnormally.

Note: Here, we are showing the try-and-catch block execution. Later in our upcoming videos, we will discuss the need and use of finally block in C#.

Example to Handle an Exception using Try-Catch Implementation with Generic Catch Block in C#

The catch block without exception class is called a generic catch and the generic catch block in C# can handle any type of exception that is raised in the corresponding try block. For better understanding, please have a look at the below example. Here, we created the catch block without any Exception class.

```
using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Number1, Number2, Result;
            try
            {
                Console.WriteLine("Enter First Number:");
                Number1 = int.Parse(Console.ReadLine());
                Console.WriteLine("Enter Second Number:");
                Number2 = int.Parse(Console.ReadLine());
                Result = Number1 / Number2;
                Console.WriteLine($"Result = {Result}");
            }
            catch
            {
                Console.WriteLine("Some Error Occurred...");
            }
            Console.ReadKey();
        }
    }
}
```

Output1: Enter the value as 10 and 0

```
Enter First Number:
10
Enter Second Number:
0
Some Error Occurred...
```

Output2: Enter the value as 10 and abc

```
Enter First Number:
10
Enter Second Number:
abc
Some Error Occurred...
```

In the above example, there is no exception class used in the try block, so it is known as the generic catch block. The problem with the generic catch block is that, any kind of exception occurs, the same message will be displayed to the end-user and the end-user cannot understand why the error has occurred; to overcome this, specific catch blocks are used. Using specific catch blocks it is possible to know more information about the exception.

Properties of Exception Class in C#:

The Exception Class is the superclass of all Exception classes and this provides some virtual properties and methods which are re-implemented by the child classes. If you go to the definition of the Exception class, then you will see the following.

```
...public class Exception : ISerializable, _Exception
{
    ...public Exception();
    ...public Exception(string message);
    ...public Exception(string message, Exception innerException);
    ...protected Exception(SerializationInfo info, StreamingContext context);

    ...public virtual string Source { get; set; }
    ...public virtual string HelpLink { get; set; }
    ...public virtual string StackTrace { get; }
    ...public MethodBase TargetSite { get; }
    ...public Exception InnerException { get; }
    ...public virtual string Message { get; }
    ...public int HResult { get; protected set; }
    ...public virtual IDictionary Data { get; }

    ...protected event EventHandler<SafeSerializationEventArgs> SerializeObjectState;

    ...public virtual Exception GetBaseException();
    ...public virtual void GetObjectData(SerializationInfo info, StreamingContext context);
    ...public Type GetType();
    ...public override string ToString();
}
```

Some of the important properties of the Exception Class are properties as follows:

1. **Message:** This property will store the reason why an exception has occurred.
2. **Source:** This property will store the name of the application from which the exception has been raised.
3. **HelpLink:** This is used to provide a link to any file /URL to give helpful information to the user about why this exception is raised.
4. **StackTrace:** This is used to provide more information about the Exception like the reason for the exception, at what method and what class the exception occurred, and at what line number the exception has occurred which helps us to resolve the issue.

Exception Handling in C# using Try-Catch Implementation with Exception Catch Block

In the below example, we have created a catch block that takes the Exception class as a parameter and within the catch block, we print the exception information using the Exception class properties i.e. Message, Source, StackTrace, and HelpLink. As you can see in the below code, we are using the super Exception class. This class is the superclass of all exception classes, so it will handle all types of exceptions raised in the try block.

```
using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Number1, Number2, Result;
            try
            {
                Console.WriteLine("Enter First Number:");
                Number1 = int.Parse(Console.ReadLine());
                Console.WriteLine("Enter Second Number:");
                Number2 = int.Parse(Console.ReadLine());
                Result = Number1 / Number2;
                Console.WriteLine($"Result = {Result}");
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Message: {ex.Message}");
                Console.WriteLine($"Source: {ex.Source}");
                Console.WriteLine($"HelpLink: {ex.HelpLink}");
                Console.WriteLine($"StackTrace: {ex.StackTrace}");
            }
            Console.ReadKey();
        }
    }
}
```

Output1: Enter the value as 10 and 0

```
Enter First Number:
10
Enter Second Number:
0
Message: Attempted to divide by zero.
Source: ExceptionHandlingDemo
HelpLink:
StackTrace:    at ExceptionHandlingDemo.Program.Main(String[] args) in D:\Projects\
ExceptionHandlingDemo\ExceptionHandlingDemo\Program.cs:line 15
```

Output2: Enter the value as 10 and abc

```
Enter First Number:
10
Enter Second Number:
abc
Message: Input string was not in a correct format.
Source: mscorlib
HelpLink:
StackTrace:    at System.Number.StringToNumber(String str, NumberStyles options, NumberBuffer
& number, NumberFormatInfo info, Boolean parseDecimal)
    at System.Number.ParseInt32(String s, NumberStyles style, NumberFormatInfo info)
    at System.Int32.Parse(String s)
    at ExceptionHandlingDemo.Program.Main(String[] args) in D:\Projects\ExceptionHandlingDemo\
ExceptionHandlingDemo\Program.cs:line 14
```

In the above example, the Exception superclass is used to handle all types of exceptions thrown from the corresponding try block. But if we use the super Exception class, when there is a relevant child exception class available, it will kill the execution performance of the program. So, in the next video, I will show you how to implement multiple catch blocks to handle different types of exceptions.

In the next article, I am going to discuss how to use **Multiple Catch Blocks and Finally Block** in C# with Examples. Here, in this article, I try to explain **Exception handling in C#** with Examples. I hope you understood how to implement Exception Handling in C# with Examples.